

Installation

Make sure that the folder containing the code, e.g. "Chemistry python simulation.py", also contains the "Inputs.xlsx" file. Use an appropriate python interpreter (e.g. IDLE, Jupyter).

Install the following python libraries: pygame, numpy, matplotlib, pandas. This will depend on how you installed python, but with pip, you can use "pip install pygame numpy matplotlib pandas".

Running

Specify inputs in the "Inputs.xlsx" file. **Specify duplicate molecules separately**, e.g. "NO+NO" rather than "2NO". Make sure to **write reactants and products in alphabetical order** (e.g. $\text{NH}_3 + \text{NO}_2$ rather than $\text{NO}_2 + \text{NH}_3$). Unique 6+ atom substances can't be generated (combinations like CH_4 and NO_3 should work), and three reactants and three products are the current maxima. There is no functionality to add charge. With these rules in mind, you can come up with whatever silly atoms, molecules and reactions you want to. To make a forwards reaction one-way, you can set K to a very high value (e.g. 50), or set it to a very low (but >0) value for a one-way backwards reaction.

You don't need to supply a k value unless you're doing a reaction involving one reactant and/or one product. The supplied k value is for the forwards reaction – supply it even if the backwards reaction is unimolecular and the forwards reaction isn't, since it is used to calculate the backwards reaction's rate. A backwards unimolecular reaction won't occur if $k = 0$. For the $\text{N} + \text{O} \rightleftharpoons \text{NO}$ reaction, you can permit NO decomposition to occur faster than the default by setting the k value to e.g. 0.1.

Save **and close** the Excel file. Run the python code, either by double clicking the file or running via the python interpreter. The simulation window will pop up. With the simulation window selected, press 'p' to pause the simulation, or press the tab key to switch to displaying Lewis structures – use the arrow keys to browse each of the different structures specified in the input folder. Press the tab key again (with the simulation window selected) to switch back to the simulation. If the setting to save invalid Lewis structures is enabled, you will see these structures flash up at the start of the simulation, and whenever they are produced for the first time. For the graph, the elapsed time at the bottom of the graph may not equal the real elapsed time (due to lag), but it will be consistent with the specified simulation parameters (e.g. the k value specified for the reaction).

Close the simulation by pressing the Escape key whilst the simulation window is selected (you may need to minimise / close the graph first). The output of "rollingData" will be stored in a "substanceCounts.xlsx" Excel file, to look at the information in each frame of the simulation. Note that only by exiting the simulation as above (pressing the Escape key, as opposed to pressing the 'X' button on the window) will "substanceCounts.xlsx" be updated. You can modify this output if you would like, by adding additional variables to frameEntry (see the code).

First-order code

This code only works for a unimolecular decomposition reaction, for a maximum of three products. You can supply the desired k value in the respective line of the "Inputs.xlsx" file.

Only one reaction at a time can be studied here, the one that you have supplied a k value for (I could add simultaneous first-order reactions if desired).

For a simple test with NO₂, you'll need to add it to Inputs.xlsx. I haven't included it originally, so as to not conflict with other reactions involving NO₂ (it is best to temporarily remove all other reactions involving the reactants and products). You will want a high K value, to avoid the reverse reaction from occurring, and a very low / zero E_a, so that every collision leads to a reaction (if few collisions lead to a reaction, then it will be more difficult to obtain a linear fit in a given time – you would need to run the simulation for longer). See the example below, with the decomposing molecule as the reactant, and no other reactions with NO₂, NO or O, alongside a low E_a and a high K:

Reaction information		(write reactants and products in alphabetical order)			
Reactants	NO2				
Products	NO+O				
K	2000				
E _a / J mol ⁻¹	1				
k / s ⁻¹ (forwards re)	0.04				
Study for second-order	N				

Note that a value of k = 0.04 s⁻¹ implies a 0.04 (1/25) probability per second of a molecule decomposing – it does not imply 0.04 molecules decomposing per second.

Second-order code

This code only works for bimolecular reactions at the moment. You can specify the reaction that you would like to study for second order kinetics in the relevant line of “Reaction information” in “Inputs.xlsx”, with “Yf” for the forwards or “Yb” for the backwards reaction. Only specify Yf/Yb for **one** reaction (all others should be “N”). If the molecules aren't identical, the kinetics are displayed in terms of the first molecule specified. In contrast to the first-order code, the second-order code does not need a k value to be provided, since the rate of reaction is just dependent on the frequency of collisions, e.g. between two NO molecules, as well as the activation energy (which can be set to 0 J mol⁻¹ for simple kinetic studies – this results in every collision leading to a reaction). You can find a full explanation at [9.7: Theories of Reaction Rates - Chemistry LibreTexts](#). Assuming an E_a of 0, the reaction rate for reactants A and B is given by the collision frequency, Z, as seen in the below equation:

$$Z = N_A N_B \sigma_{AB} \sqrt{\frac{8 k_B T}{\pi \mu_{AB}}}$$

where *N* is the number of molecules, *σ* is the collision cross-section of the two molecules, and *μ* is the reduced mass. Therefore, you can indirectly alter k by changing the temperature, the radius of the atoms, the size of the simulation, etc. An example for NO (as shown in the video) is specified below – here, the large E_a for the forwards reaction helps to prevent the forwards reaction from occurring. Since the value of K is very low, the reverse E_a is 0 (as per the Gibbs energy). Remember to stick to the start of the simulation, when the linear fit is the strongest (you can pause the simulation by pressing ‘p’ with the simulation window selected). As with the first-order code, it is best to just have one reaction containing the reactants and products in the ‘Reaction information’ section of Inputs.xlsx.

Reaction information	
Reactants	N2+O2
Products	NO+NO
K	0.00001
Ea / J mol ⁻¹	10000
k / s ⁻¹ (forwards)	
Study for second-order	Yb

Arrhenius plot data generation

This code is specific for $\text{N}_2 + \text{O}_2 \rightarrow 2\text{NO}$. An equal number of N_2 and O_2 molecules are needed. The “temperatureOptions” list gives the possible temperatures, and the “for trial in range(20)” loop gives the total number of runs. The output is saved in the file “k_vs_temp.xlsx”, which you can then use to make the Arrhenius plot. For this to work well, a high E_a is needed (so that it is primarily the E_a , rather than the collision frequency, that is determining reaction rate), and as above, you should specify one reaction only, and provide a K value that gives an effectively one-way reaction. An example is below.

Reaction information	
Reactants	N2+O2
Products	NO+NO
K	2000
Ea / J mol ⁻¹	10000
k / s ⁻¹ (forwards)	
Study for second-order	N

Expected amounts of each species at equilibrium

See the “Equilibrium expected amounts” code. This code can solve equations for 1 or 2 equilibrium reactions, using the fsolve() function. What fsolve() is doing is trying to find the roots of an equation (the points when the equation is equal to 0). It’s best to try and read through the code to understand what it’s doing, but e.g. for the reaction $\text{N}_2 + \text{O}_2 \rightarrow 2\text{NO}$, it takes

$$\frac{[\text{NO}]^2}{[\text{N}_2][\text{O}_2]} - K_{\text{actual}}$$

and tries to make this equal to 0. It does this by taking the initial values of [NO], [N₂] and [O₂] and adjusts these until equation (1) is equal to 0; this implies that [NO], [N₂] and [O₂] are at the amounts needed to make [NO]²/[N₂][O₂] equal to K_{actual} , and hence these are the equilibrium values.

For this code, the equations are defined at the top (these just take reactants and products, rather than particular species), followed by the solving functions, followed by the variable definitions, function calls and prints. Please read through each line, to understand what the

code is doing – it may be useful to print out variables such as `initialAmounts`, and to bear in mind the functions at the top of the code.